

Углубленное изучение архитектуры Solana

Занятие 2



Кафедра блокчейна

План занятия

Лекция (теория)

- Структура сети Solana: валидаторы, кластеры и шреды
- Модель данных Solana и поток транзакций
- Валидация транзакций и распространение блоков
- Преимущества производительности Solana по сравнению с другими блокчейнами (сравнение с Ethereum и Bitcoin)
- Роль Proof of History (PoH) в улучшении масштабируемости и пропускной способности
- Base fee per transaction/Compute Units/Storage rent/Priority Fee
- Что такое лампорты ($1 \text{ SOL} = 1e9 \text{ lamports}$)

Семинар (практика)

- Рисуем в draw.io: Program, PDA, System Program, Account, Signer, Pubkey
- CLI: смотрим блоки, транзакции, логи и состояние сети

Структура сети: валидаторы

Кто такой валидатор в Solana

- Держит копию ledger и исполняет транзакции в runtime
- Участвует в консенсусе PoS (Tower BFT) — голосует за слоты/блоки
- По расписанию становится лидером (producer) и формирует блок
- Распространяет данные блока через Turbine (шреды)
- Публикует состояние через Gossip и обслуживает клиентов (если это RPC-узел)

Роли в сети

Leader (слот)

Leader (слот)

Leader (слот)

Leader (слот)

Структура сети: кластеры

Кластер = набор валидаторов с общим genesis и единой историей

Devnet

- Публичный стенд для разработки
- Airdrop test SOL
- Поведение близко к mainnet

Testnet

- Стресс-тесты валидаторов
- Может быть reset ledger
- Часто более новая версия

Mainnet-beta

- Прод
- Реальные SOL и стоимость
- Наиболее строгие требования

<https://solana.com/docs/references/clusters>

Важно: транзакции не синхронизируются между кластерами — это отдельные цепочки.

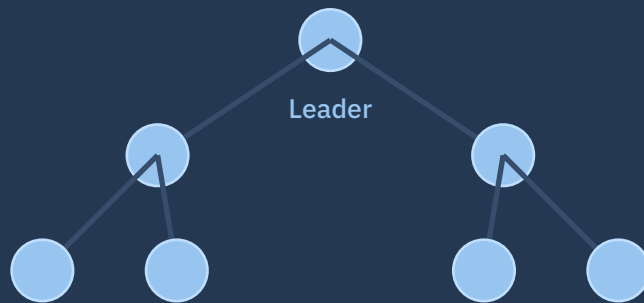
Turbine: шреды (shreds)

Solana не использует execution-sharding как L1

- Яковенко работал на MediaFLO и опирался на BitTorrent
- Состояние не бьется на шарды. Единое глобальное состояние (accounts) в одном кластере
- Разделение данных для передачи — это «шреды» (части блока), а не шардинг состояния
- Масштабирование не через L2 или шардинг, а параллельное исполнение (Sealevel) + параллельное распространение шард (fanout)

Что такое шреды

- Блок режется на фиксированные куски (~под MTU UDP)
- Есть data shreds и coding shreds (Reed–Solomon erasure coding)
- Рассылка по дереву Turbine снижает нагрузку на лидера
- Получатель восстанавливает блок, даже если часть шредов потеряна



Reed-Solomon Error Correction Codes

Избыточное формирование сообщений для коррекции возможных ошибок

Дано:

Алфавит состоит из целых чисел.

Размер k слова-сообщения равен 4.

Возьмем 2 избыточных символа, поэтому длина кодового слова n равна 6.

Полином $p(x)$ всегда вычисляется для следующих значений x : $(-1, 0, 1, 2, 3, 4)$. Обратите внимание, что значений x столько же, сколько символов в кодовом слове.

Пример обработки

1) Пусть дано слово $[2, 3, -5, 1]$

2) Вычисляем $p(x) = 2 + 3x - 5x^2 + x^3$ на заранее выбранных значениях x и формируем кодовое слово

3) Кодовое слово: $[7, 2, 1, -4, -7, -2]$. Отправляем его

4) Декодер получает $[7, 2, 1, -4, -7, -2]$ и восстанавливает исходные пары $(-1, 7), (0, 2), (1, 1), (2, -4), (3, -7), (4, -2)$

5) Решая примитивную систему линейных уравнений, декодер получает оригинальные коэффициенты $[2, 3, -5, 1]$

Исправление ошибок

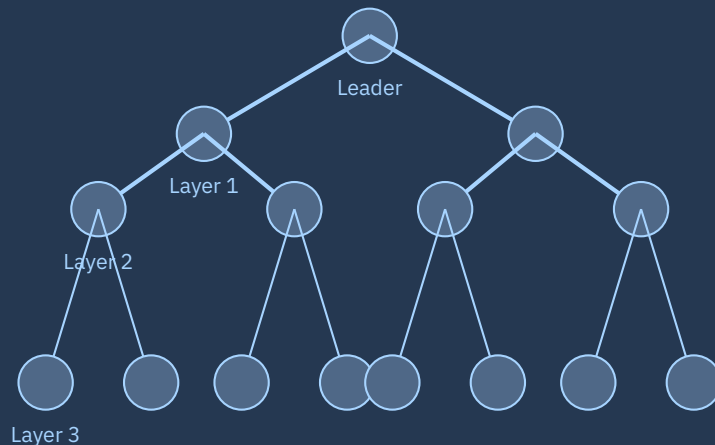
Не сходятся решения? Перебираем все возможные четверки и берем решение, которое встречается чаще всего. На N ошибок необходима избыточность $2N$. На 6 букв - 15 уникальных четверок.

Валидация транзакций и распространение блоков

Проверки и консенсус

- Проверка подписи + account locks (read/write) перед исполнением
- Runtime исполняет инструкции, считая «budget» Compute Units
- Результат фиксируется в entries/blocks; валидаторы голосуют (голос - тоже транзакция)
- Лидер заинтересован положить голоса в блок, ведь это закрепляет вес форка
- Подтверждение строится на PoS + Tower BFT

Turbine: рассылка шредов



Дерево уменьшает «one-to-all» рассылку.
Erasure coding повышает устойчивость.

Gulf Stream

Форвардинг транзакций к лидерам ближайших слотов (без «классического мемпула»)

Зачем это нужно:

Мемпул исключен как слабое звено в масштабировании. Валидаторы пересылают tx заранее по расписанию лидеров. Tx имеет привязку к конкретному блоку, и у нее есть 32 блока на исполнение. Будущий лидер держит tx в очереди TPU → быстрее заполняет слот, меньше latency до включения. кто платит выше Priority Fee и не конфликтует по read/write-lock'ам аккаунтов, а остальные будут отложены/вытеснены

Как это выглядит в пайплайне:

- 1) Клиент/Wallet → RPC отправляет tx валидатору
- 2) Валидатор отправляет tx к будущим лидерам
- 3) Лидер слота исполняет (TPU/Sealevel) → формирует блок
- 4) Если лидер не успел исполнить, tx исполнит следующий лидер
- 5) Лидер всегда слушает предыдущего лидера
- 6) Turbine разносит блок по сети → голоса (Tower BFT)

Клиент / Wallet RPC

Будущий лидер (очередь TPU)

Будущий лидер (очередь TPU)

Модель данных Solana (Accounts + Programs)

Account = запись состояния

- Адрес (Pubkey) → данные + lamports + owner
- Программа-владелец (owner) единственная может менять data и списывать lamports
- Program account содержит код (executable), state хранится отдельно в data accounts
- Rent: для хранения данных нужен минимум баланса (rent-exempt)

Типа аккаунтов

Pubkey	Owner	Lamports	Data
Wallet A	System	...	-
Program P	BPF Loader	rent-exempt	code
PDA	Program P	rent-exempt	bytes
Token acct	SPL Token	rent-exempt	bytes

Раньше предполагалась аренда (rent), но от этого отказались

PDA (program data account)

PDA — это адрес аккаунта, который детерминированно вычисляется из:

- seeds[] (байтовые строки),
- Program_id,
- bump.

PDA не имеет приватного ключа.

PDA может подписывать только своя программа и только во время исполнения (через `invoke_signed`)

```
PDA = find_program_address(seeds, program_id)
```

```
#[account(
  init,
  seeds = [b"user", user.key().as_ref()],
  bump,
  payer = user,
)]
pub user_state: Account<'info, UserState>;
```

Sealevel, Cloudbreak

Порядок создания программы

- 1) Создайте новый public key
- 2) Переведите монеты на этот адрес (плата за память)
- 3) Укажите System Program выделить память.
- 4) Укажите System Program передать владение аккаунтом Loader'у.
- 5) Загрузите байт-код в память (используются промежуточные аккаунты-буферы)
- 6) Укажите программе-загрузчику пометить память как исполняемую.
- 7) Loader проверяет байткод, и теперь этот байткод можно использовать как программу

Правила сети

- 1) Программы могут изменять только данные принадлежащих им аккаунтов
- 2) Программы могут списывать средства только с принадлежащих им аккаунтов.
- 3) Любая программа может зачислять средства на любой аккаунт.
- 4) Любая программа может считывать данные с любого счета.

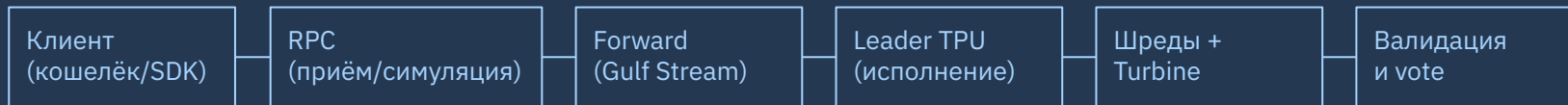
Оптимизация транзакций

Программа ЗАРАНЕЕ перечисляет все аккаунты, которые она будет использовать (readonly или mutable)

Поток транзакций: от клиента до блока

Ключевые механизмы:

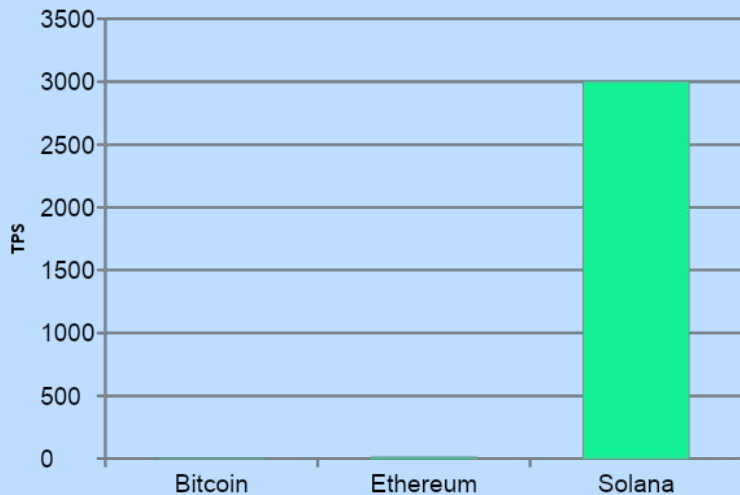
- Нет классического mempool: транзакции заранее форвардятся к будущим лидерам (Gulf Stream)
- Исполнение в leader TPU → затем блок режется на шреды и быстро разносится (Turbine)



Скорость Solana: сравнение с Ethereum и Bitcoin

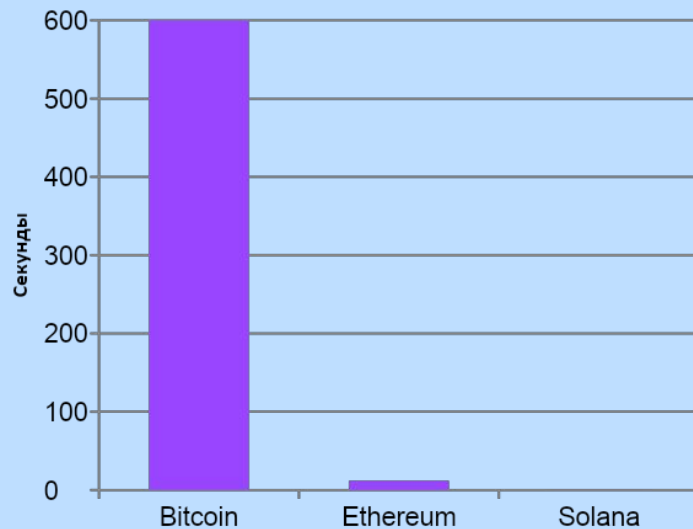


Пропускная способность (L1, типично)



Пояснение: порядок величин; у Solana часть tx — голоса валидаторов.

Время блока / слота (примерно)



Solana: слот ~400мс (может колебаться).

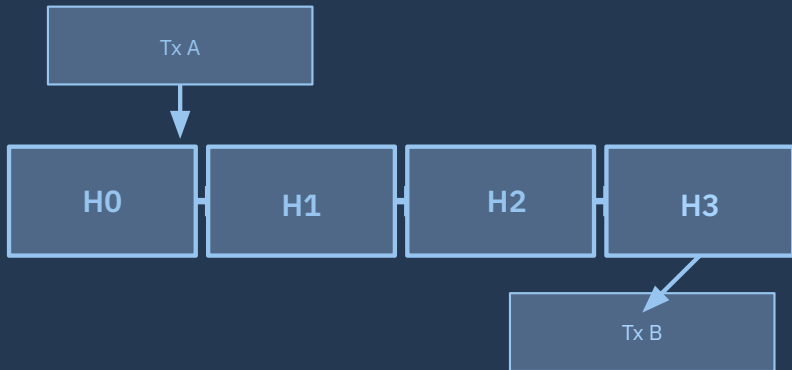
Proof of History (PoH): зачем он нужен

Интуиция

- PoH — криптографическая «линейка времени»: последовательная цепочка хэшей
- Даёт доказуемый порядок событий без постоянного общения всех со всеми
- Упрощает работу BFT-консенсуса: не нужно согласовывать однозначный порядок транзакций

Важно: PoH сам по себе не «консенсус», а механизм упорядочивания/таймстампов.

Схема (упрощённо)



Каждый следующий хэш зависит от предыдущего → «время» нельзя ускорить, но легко проверить.

Экономика транзакций: из чего складывается стоимость

Base fee

по подписям

5000 лампоров за
каждую подпись в
транзакции

50% burn, 50% валидатору

Compute

Compute Units

лимит CU
+ опционально
price за CU

задаётся Compute Budget
Program

Storage

rent-exempt

минимальный
баланс для
хранения data

рассчитывается
по размеру data

Priority

ускорение

приоритет
через price
(μ -lamports/CU)

чем выше — тем выше шанс
включения

Единица: 1 SOL = 1 000 000 000 лампоров (1e9).

Солана - не дефляционный токен. Валидаторы получают награду, которая печатается из воздуха. Изначальная инфляция 8% уменьшается на 15% в год, пока не снизится до 1.5% в год

Base fee per transaction

Базовая комиссия

- 5000 лампортов за каждую подпись (signature) в транзакции
- Плательщик — первый signer (fee payer) в сообщении
- Только аккаунты System Program могут платить комиссии
- Базовая комиссия делится: 50% burn, 50% валидатору

Пример

Минимальная часть стоимости

```
1 1 подпись → 5_000 lamports
2 2 подписи → 10_000 lamports
3 3 подписи → 15_000 lamports
4
5 fee_payer = signatures[0]
```

На перегруженной сети базовая комиссия — это «входной билет», а конкуренция идёт через priority fee.

Compute Units (CU): «газ» по-солановски

Что важно помнить

- Каждая инструкция потребляет CU при исполнении в runtime
- Есть лимит CU на транзакцию; его можно поднять через Compute Budget Program
- Цена за CU по умолчанию 0 (если не выставить priority fee)
- Параллельность: транзакции с непересекающимися write-lock аккаунтами могут выполняться одновременно (Sealevel)

Как задать CU limit/price

Compute Budget Program

```
1 // Вставить в начало транзакции
2 ComputeBudgetProgram.setComputeUnitLimit({ units:
  400_000 })
3 ComputeBudgetProgram.setComputeUnitPrice({
  microLamports: 5_000 })
4
5 // priorityFee = microLamports * units / 1_000_000
```

μ -lamports = 10^{-6} лампорта. Цена указывается в μ -lamports/CU.

Storage rent: стоимость хранения данных

Идея rent-exempt

- Аккаунт хранит данные в сети → это ресурс (storage)
- Чтобы аккаунт не «исчез», обычно пополняют его до уровня rent-exempt
- Минимум зависит от размера data (bytes)
- Рассчитывается через RPC:
`getMinimumBalanceForRentExemption`

Практика

Оценка минимального баланса

```
1 # Узнать rent-exempt для аккаунта данных
2 solana rent 165
3
4 # Через RPC (пример, curl)
5 # getMinimumBalanceForRentExemption(165)
```

В программах: размер data = ваш layout/сериализация → влияет на стоимость и UX.

Priority Fee и лампорты

Priority fee

Опциональная комиссия, чтобы «обогнать очередь» в слоте лидера

Задаётся через SetComputeUnitPrice (μ -lamports/CU)

Обычно ставят вместе с SetComputeUnitLimit

Total fee в meta.fee включает base fee + compute-unit fee

Лампорты

**1 SOL = 1 000 000 000
лампортов (1e9)**

Мини-пример

1 # Узнать rent-exempt для аккаунта данных

2 solana rent 165

3

4 # Через RPC (пример, curl)

5 # getMinimumBalanceForRentExemption(165)

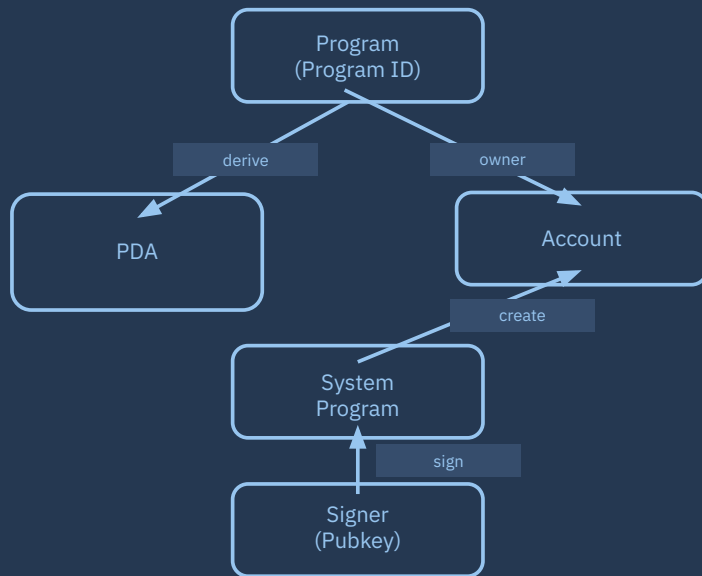
Семинар: рисуем схему в draw.io



Что рисуем

- Program (ваш on-chain код) и его Program ID
- Account (данные/балансы) и owner
- System Program: create_account, transfer, assign owner
- PDA: адрес, вычисленный из seeds + programId (без приватного ключа)
- Signer: аккаунт, который подписывает транзакцию
- Pubkey: адрес аккаунта (ключ в «таблице accounts»)

Подсказка по связям (стрелки)



Семинар: Solana CLI — блоки, транзакции, логи



Быстрый чеклист

Команды для демо

1) Выбор кластера

```
solana config set --url https://api.devnet.solana.com
```

2) Слоты и блоки

```
solana slot
```

```
solana block <SLOT>
```

3) Транзакция

```
solana confirm <SIGNATURE> -v
```

4) Сеть

```
solana validators
```

```
solana gossip
```

5) Логи (нужен websocket endpoint/RPC)

```
solana logs
```

Что смотреть в выводе

- В block: time, parent slot, tx count, rewards
- В confirm -v: logs, inner instructions, compute units, meta.fee
- В logs: сообщения программ и runtime (для отладки)
- В validators: состояние синхронизации и версия ПО

Итоги и вопросы

Если после занятия вы можете:

Объяснить разницу
«accounts vs programs»

Описать путь транзакции и роль
leader / Turbine / votes

Прикинуть комиссии (base fee + priority)

Нарисовать взаимосвязи
Program/PDA/System/Program/Account/Signer

...ТО ВЫ ГОТОВЫ К следующим
темам: CPI, Anchor, SPL Token.